南方科技大学

LAB1: Python Review

COE201, Spring 2026

2026-02-28

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 609

Outline

Lab Courses Intro 1

About COE 201 2

Python Review 4

Why Python 5

Python Setup 6

Python Basics 9

Practical Python Tips 23

Other Great References 28

Vectorization & NumPy Basics 29

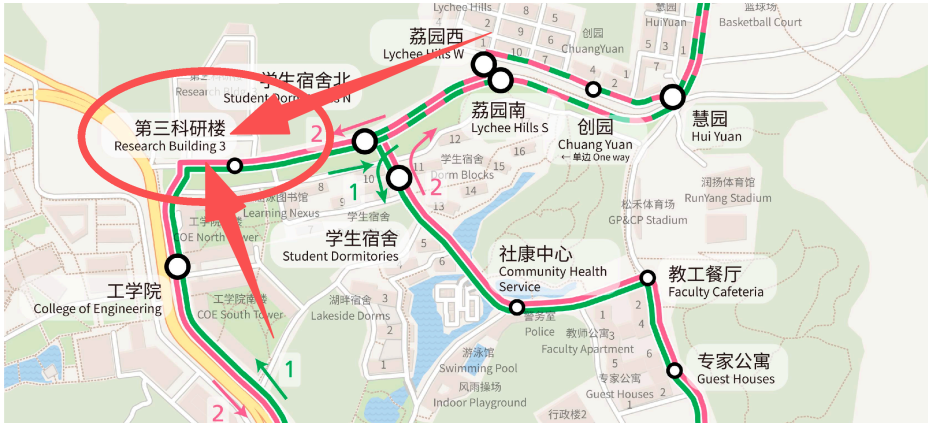
Lab Tasks 33

Lab 1 Exercises 34

Lab Courses Intro

About COE 201

- COE201 topic: **Large Model** from scratch
- About me: Peng Weiyuan (彭维源), RA @ HQ Lab
- Office hour: every Thur, 14:30 - 15:30 (except for public holidays)
  - Find me @ 3rd Research Building (三科), Room 609



- No separate QQ group for the lab course (use the main course group)

About COE 201

Lab Courses LLM Usage Policy

Since COE201 is a second-year course, we won't make the labs overly difficult, so take it easy.

LLM usage is allowed, but follow this order when you get stuck:

1. **Try it yourself first** - spend some time debugging and thinking
2. **Ask the TA or classmates** - discuss the problem, we're here to help!
3. **Then use LLM** - if you still need assistance

This helps you actually learn rather than just getting answers.

Python Review

Why Python

- Widely used, general purpose
- Easy to learn, read, and write
- Used by major deep learning and LLM frameworks (PyTorch, Transformers)
- Active open-source, many libraries!

Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		Python	22.61%	-0.68%
2	**4**	^	**C**	**10.99%**	**+2.13%**
3	3		Java	8.71%	-1.44%
4	2	v	C++	8.67%	-1.62%
5	5		C#	7.39%	+2.94%
6	6		JavaScript	3.03%	-1.17%
7	9	^	Visual Basic	2.41%	+0.04%
8	8		SQL	2.27%	-0.14%
9	11	^	Delphi/Object Pascal	1.98%	+0.19%
10	18	^^	R	1.82%	+0.81%

Figure 1: TIOBE Index for January 2026

Python Setup

Environments Management

Problem

- Different versions of Python, e.g. 3.11, 3.9 ...
- Countless Python packages and their dependencies
- Different projects require different packages

Even worse, different versions of the same package!

Solution

- Keep multiple Python environments that are isolated from each other
- Each environment
  1. Can use different Python version
  2. Keeps its own set of packages (can specify package versions)

Python Setup

Manage Environments

We don't want to spend excessive time on environment setup, **but this is an important skill for LLM development.**

`uv` is a modern solution for Python environment management. Install from https://docs.astral.sh/uv/getting-started/installation.

Basic Workflow:

- Install Python: `uv python install python3.12`
- Initialize new `pyproject.toml`: `uv init`
- Add packages: `uv add 'numpy>2.0'`
- Use existing `pyproject.toml`: `uv sync`

Python Setup

IDEs / text editors for Python

- PyCharm
- Visual Studio Code
- Vim (for Linux or Mac) ...

Write a Python program in your IDE or text editor of choice.

Run a Python program in terminal:

```shell
# Activate virtual env (venv) and run program with command
source .venv/bin/activate  # if using bash-compatible shell
python <filename.py>
deactivate  # deactivate current venv
# or use `uv run` directly (do not need venv activated):
uv run <filename.py>
```

Basic Data Types - Numbers

Integers and floats work as you would expect from other languages:

```python
# Integer operations
x = 3
print(x)                        # OUTPUT: 3
print(x + 1, x - 1)             # OUTPUT: 4 (addition) 2 (subtraction)
print(x * 2, x ** 2)            # OUTPUT: 6 (multiplication)  9 (exponentiation)
# Augmented assignment
x += 1                  # x becomes 4
print(x)                # OUTPUT: 4
x *= 2                  # x becomes 8
print(x)                # OUTPUT: 8
# Float operations
y = 2.5
print(y, y + 1, y * 2, y ** 2)  # OUTPUT: 2.5 3.5 5.0 6.25
```

Basic Data Types - Strings

```python
# Basic operations
s1, s2 = "hello", "world"
print(s1 + " " + s2)          # OUTPUT: hello world (concatenation)
print(s1 * 3)                 # OUTPUT: hellohellohello (repetition)

# Indexing and slicing
s = "hello world"
print(s[0])                   # OUTPUT: h (first character)
print(s[-1])                  # OUTPUT: d (last character)
print(s[0:5])                 # OUTPUT: hello (slicing)
print(s[::-1])                # OUTPUT: dlrow olleh (reverse)

# Common methods
text = "  Hello, World!  "
print(text.strip())           # OUTPUT: Hello, World! (remove whitespace)
```

Python Basics

```python
print(text.replace("World", "Python"))  # OUTPUT:   Hello, Python!  (replace)
print(text.split(","))        # OUTPUT: ['  Hello', ' World!  '] (split into list)

# Strings are immutable - this would cause TypeError
# text[0] = "H"               # TypeError: 'str' object does not support item assignment

# F-strings (formatted string literals)
name, age = "Alice", 30
print(f"Name: {name}, Age: {age}")       # OUTPUT: Name: Alice, Age: 30
print(f"Next year: {age + 1}")           # OUTPUT: Next year: 31
pi = 3.1415926
print(f"PI to 3 decimals: {pi:.3f}")     # OUTPUT: PI to 3 decimals: 3.142
```

Data Container — Lists

`list` is appendable and can contain elements of different types:

```python
# Creating and accessing lists
xs = [3, 1, 2]            # Create a list
print(xs)                 # OUTPUT: [3, 1, 2]
print(xs[2])              # OUTPUT: 2 (access by index)
print(xs[-1])             # OUTPUT: 2 (negative index from end)

# Lists are mutable
xs[2] = 'foo'             # Modify element
print(xs)                 # OUTPUT: [3, 1, 'foo']

# List methods
xs.append('bar')          # Add element to end
print(xs)                 # OUTPUT: [3, 1, 'foo', 'bar']
```

Python Basics

```python
# Loop over a list
xs = [3, 1, 'foo', 'bar']
for x in xs:
    print(x)             # OUTPUT: 3 (then 1, 'foo', 'bar' on separate lines)

# Slicing: list elements can be accessed in convenient ways
# Basic format: some_list[start_index:end_index]
numbers = [0, 1, 2, 3, 4, 5, 6]
numbers[0:3] == numbers[:3] == [0, 1, 2]
numbers[5:] == numbers[5:7] == [5, 6]
numbers[:] == numbers == [0, 1, 2, 3, 4, 5, 6]
numbers[-1] == 6 # Negative index wraps around
numbers[-3:] == [4, 5, 6]
numbers[3:-2] == [3, 4] # Can mix and match
```

Data Container — Tuples

`tuple` is immutable and can contain elements of different types:

```python
# Creating tuples
t = (1, 2, 3)           # Parentheses recommended
single = (1,)           # Single-element tuple (note comma)
empty = ()              # Empty tuple

# Indexing and slicing
print(t[0])             # OUTPUT: 1
print(t[1:3])           # OUTPUT: (2, 3) (slicing)
print(len(t))           # OUTPUT: 3

# Tuples are immutable
# t[0] = 10             # TypeError: 'tuple' object does not support item assignment
```

Python Basics

```python
# Tuple unpacking
x, y, z = t             # Unpack tuple into variables
print(x, y, z)          # OUTPUT: 1 2 3

# Swapping variables using tuple unpacking
a, b = 10, 20
a, b = b, a             # Swap values
print(f"a={a}, b={b}")  # OUTPUT: a=20, b=10
```

Data Container — Dictionaries

A dictionary stores (key, value) pairs:

```python
# Creating and accessing dictionaries
d = {'cat': 'cute', 'dog': 'furry'}  # Create dictionary
print(d['cat'])                      # OUTPUT: cute (access by key)
print('cat' in d)                    # OUTPUT: True (check key existence)

# Adding and modifying entries
d['fish'] = 'wet'                    # Add new key-value pair
print(d['fish'])                     # OUTPUT: wet

# Safe access with get() method
# print(d['monkey'])                 # KeyError: 'monkey' not a key of d
print(d.get('monkey', 'N/A'))        # OUTPUT: N/A (default when key missing)
print(d.get('fish', 'N/A'))          # OUTPUT: wet (key exists)
```

```python

# Removing entries
del d['fish']                        # Delete key-value pair
print(d.get('fish', 'N/A'))          # OUTPUT: N/A (key no longer exists)

# Dict keys must be hashable (immutable types like str, int, tuple)
# d[[1, 2]] = "list"    # TypeError: unhashable type: 'list'
d[(1, 2)] = "tuple works!"  # Tuples are hashable
```

Functions

Functions are defined using the `def` keyword. For example:

```python
# Simple function with default argument
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Alice"))           # OUTPUT: Hello, Alice!
print(greet("Bob", "Hi"))       # OUTPUT: Hi, Bob!

# Recursive function (calls itself)
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
print(factorial(5))    # OUTPUT: 120 (5! = 5*4*3*2*1)
```

Classes

Basic class definition:

```python
# Simple class with constructor and method
class Person:
    def __init__(self, name, age):
        self.name = name   # Instance variable
        self.age = age     # Instance variable

    def greet(self):
        return f"Hello, my name is {self.name}"

# Create instance and call method
p = Person(name="Alice", age=30)
print(p.greet())          # OUTPUT: Hello, my name is Alice
```

File I/O

Basic file operations in Python. Common open modes:

- `"r"` - Read (default), file must exist
- `"w"` - Write, creates new file or **overwrites** existing
- `"a"` - Append, adds to end of file
- `"r+"` - Read and write

```python
# Writing to a file
with open("example.txt", "w") as f:
    f.write("Hello, World!\n")

# Reading from a file
with open("example.txt", "r") as f:
    content = f.read()
```

Python Basics

```python
    print(content)              # OUTPUT: Hello, World!

# Reading line by line
with open("example.txt", "r") as f:
    for line in f:
        print(f"Line: {line.strip()}")  # OUTPUT: Line: Hello, World! etc.

# Appending to a file
with open("example.txt", "a") as f:
    f.write("Appended line.\n")
```

Import Package Modules

Use keyword `import` to make use of package modules in your script:

```python
# Import ‘os’ and ‘time’ standard library modules
import os, time

# Import under an alias
import numpy as np
np.dot(x, y) # Access components with pkg.fn
# Import specific submodules/functions
from numpy import linalg as la, dot as matrix_multiply
# Be careful: importing many names directly can cause conflicts
```

Practical Python Tips

List Comprehensions

Similar to `map()` from functional programming languages (readability + succinct)

Format: `[func(x) for x in some_list]`

```python
squares = []
for i in range(10):
    squares.append(i**2)

# which is equivalent to
squares = [i**2 for i in range(10)]
```

Can use with `if`:

```python
odds = [i**2 for i in range(10) if i%2 == 1]
```

Using pathlib.Path

Modern way to handle file paths (cross-platform, more readable than `os.path`):

```python
from pathlib import Path

# Create path objects
data_dir = Path("data")
file_path = data_dir / "output" / "result.txt"  # Use / to join paths

# Common operations
file_path.exists()      # Check if path exists
file_path.is_file()     # Check if it's a file
file_path.parent        # Get parent directory: data/output
file_path.name          # Get filename: result.txt
file_path.stem          # Get name without extension: result
file_path.suffix        # Get extension: .txt
```

Practical Python Tips

```python

# Read/write files directly
file_path.write_text("Hello!")
content = file_path.read_text()

# List files in directory
for f in Path(".").glob("*.py"):
    print(f)            # All .py files in current dir
```

Debugging Tools

Debugging is a critical skill! Here are three approaches:

1. Print Debugging (Simple but effective)

```python
print(f"DEBUG: x = {x}, type = {type(x)}")  # Check variable values
print(f"DEBUG: reached line 42")             # Check code flow
```

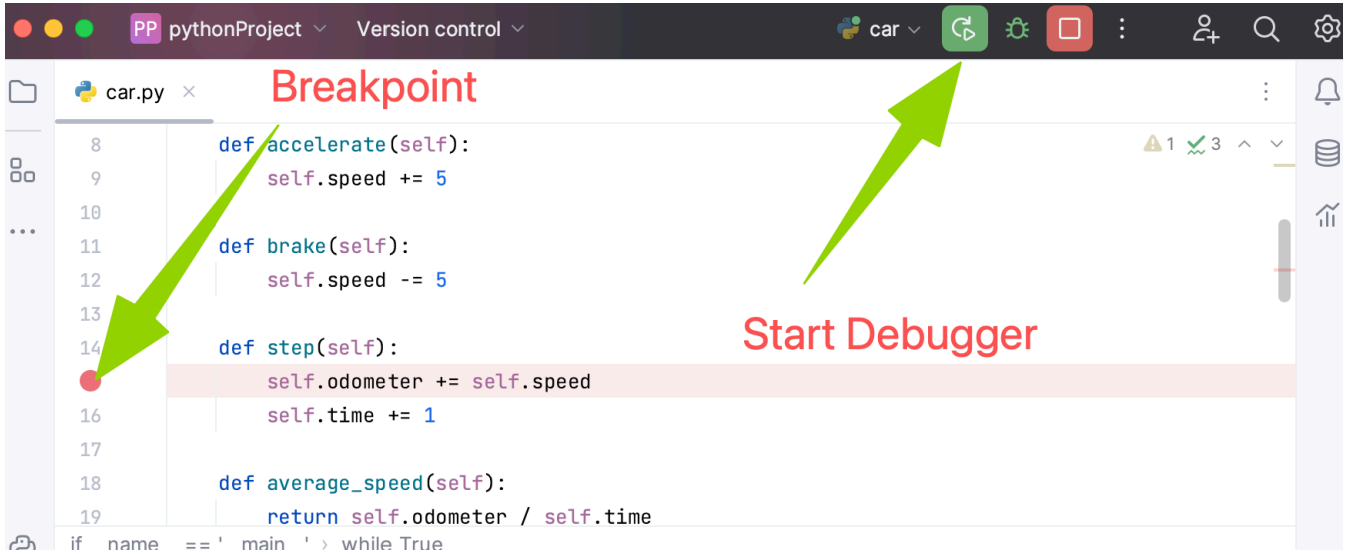
2. pdb (Command-line debugger)

```python
import pdb; pdb.set_trace()  # Insert where you want to pause
# Common commands: n (next), s (step into), c (continue), p var (print)
```

Docs: https://docs.python.org/3.12/library/pdb.html

3. IDE Debugger (Recommended for complex issues)

Set breakpoints, step through code, inspect variables. PyCharm example:



Other Great References

1. Official Python 3 documentation: https://docs.python.org/3/
   - If you encounter any language/standard library problems
2. Python Official Tutorial: https://docs.python.org/3/tutorial/
3. uv "cheat sheet": https://docs.astral.sh/uv/getting-started/features/
4. Python/Numpy tutorial: https://cs231n.github.io/python-numpy-tutorial/

Several Python and library-specific (ex. NumPy) "Cheat Sheet" guides online as well!

Vectorization & NumPy Basics

Why Vectorization?

Deep learning models involve massive amounts of data and calculations, so **high performance is essential!**

Instead of using slow Python `for` loops, we use NumPy to perform operations on entire arrays at once.

Basic NumPy Operations

```python
import numpy as np

# Create an array from a Python list
a = np.array([1, 2, 3])
print(a)           # OUTPUT: [1 2 3]

# Create a 2x2 matrix filled with zeros
z = np.zeros((2, 2))
print(z)           # OUTPUT: [[0. 0.] [0. 0.]]

# Mathematic operations are applied element-wise automatically
b = np.array([4, 5, 6])
print(a + b)       # OUTPUT: [5 7 9] (Addition)
print(a * 2)       # OUTPUT: [2 4 6] (Scalar multiplication)
```

Vectorization & NumPy Basics

```python
# Dot product (Matrix multiplication)
print(np.dot(a, b))# OUTPUT: 32 (1*4 + 2*5 + 3*6)

# Common aggregation functions
c = np.array([-1.5, 2.0, 3.5])
print(np.sum(c))   # OUTPUT: 4.0 (Sum of all elements)
print(np.max(c))   # OUTPUT: 3.5 (Find the maximum value)
print(np.abs(c))   # OUTPUT: [1.5 2.  3.5] (Absolute value)
```

Broadcasting

NumPy automatically aligns dimensions when performing operations on arrays of different shapes.

```python
import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Shape: (3, 3)
b = np.array([1, 1, 1])                         # Shape: (3,)

# Broadcasting expands vector 'b' to match matrix 'A'
# Internally, 'b' is virtually duplicated to match A's shape:
# [[1,1,1],
#  [1,1,1],
#  [1,1,1]]
C = A + b
```

Lab Tasks

Lab 1 Exercises

Complete the following tasks:

1. **Task 0: Environment Check** (Install uv, sync environment then run `lab1_env_check.py`)
2. **Task 1: Matrix Multiplication Benchmark** (Experience the speedup of vectorization compared to nested loops in `lab1_benchmark.py`)
3. **Task 2: Vectorized Softmax** (Implement Softmax using NumPy broadcasting without any loops in `lab1_math.py`)

$$S(x_i) = \frac{e^{\{x_i\}}}{\sum\left(e^{\{x_j\}}\right)}$$

Thanks for Listening!